



University
of Glasgow

W9-04: Software Sandboxes

COMPSCI4064 (H) and COMPSCI5088 (M)

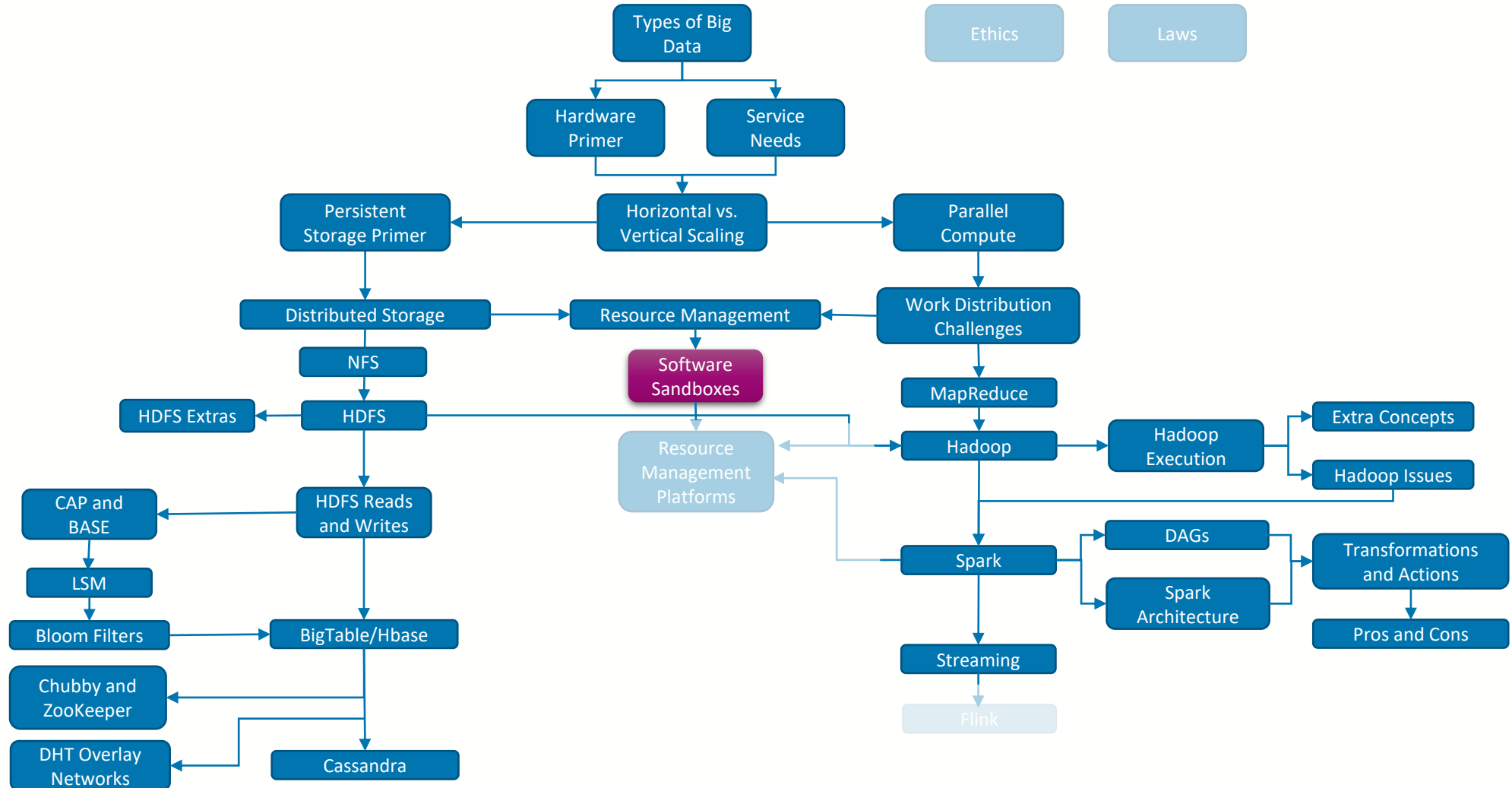
Dr. Richard McCreddie

**WORLD
CHANGING
GLASGOW**

**A WORLD
TOP 100
UNIVERSITY**



Where are we?





Incompatibilities

- One of the key problems that we run into when deploying complex software is incompatibilities with regard to **dependencies of the software**
- Incompatibilities may happen at different levels
 - **Application Level**: Two applications are running that share classes, but use different versions of that class/package the class comes from
 - **Library Level**: Two applications use packages that are incompatible with one another because they both define the same object
 - Java Logging library's are a great example of this
 - **Executable Level**: Two applications run on or use another executable, but require different versions of that executable
 - Think Java versions, or the Hadoop executables that Spark uses
 - **Driver Level**: Two applications require specific drivers to communicate with the hardware, but were written with different versions in mind
- The more software that we need to run on a single machine, the more likely we are going to experience incompatibilities
 - ... and incompatibilities can be some of the most difficult to solve issues, as it may involve editing the dependencies themselves to resolve the incompatibility



Sandboxes

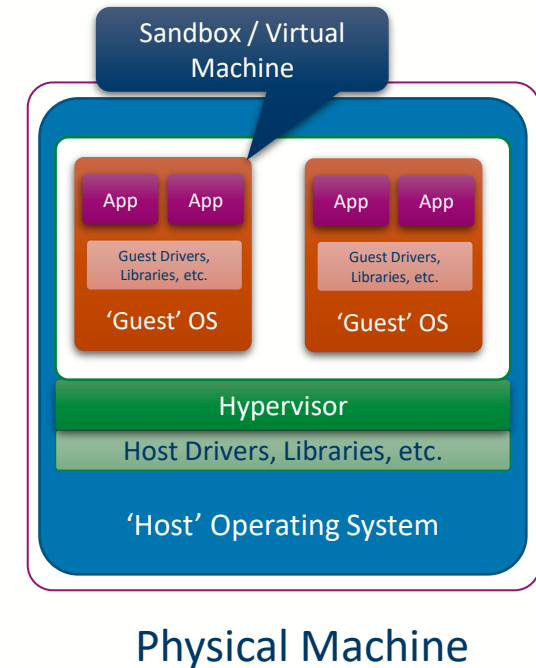
- This is not a new problem, indeed the Java Virtual Machine that you use is itself an attempt to solve **application** and **library** level incompatibilities
 - When the JVM starts, it forms a sandbox for running compiled Java code
 - Only code that is on the Class Path can be run in the sandbox
 - The JVM will only load a single version of any class
 - Creating a 'runnable jar' builds an all-in-one bundle of java code, where only classes within that jar will be used
 - The creation of such a jar will select which version of a class to use if there are multiple
- This idea of **sandboxing** or **isolating** a piece of software is powerful, as we can have multiple concurrent sandboxes running simultaneously
 - E.g. multiple JVMs running their own unique runnable jars





(System) Virtual Machines

- Ok, so a JVM (and other class management systems like it) try to solve application and library level incompatibilities, but what about executable or driver level dependencies?
- Enter **(System) Virtual Machines**
 - A Virtual Machine is a software program or operating system that exhibits the **behaviour of a separate computer** (such as running its own programs)
 - Usually, this involves running an **entire operating system with an associated file system** within a **sandbox**
 - In this way, all the application code, library's, executables and drivers are all packaged together into the sandbox
 - This package is called the **virtual machine image**





(System) Virtual Machines

Advantages

- Solves incompatibilities completely (each VM is a functionally complete system)
- VM images can be shared amongst different machines so long as there is a compatible hypervisor
 - ‘entire computer in one file’

Disadvantages

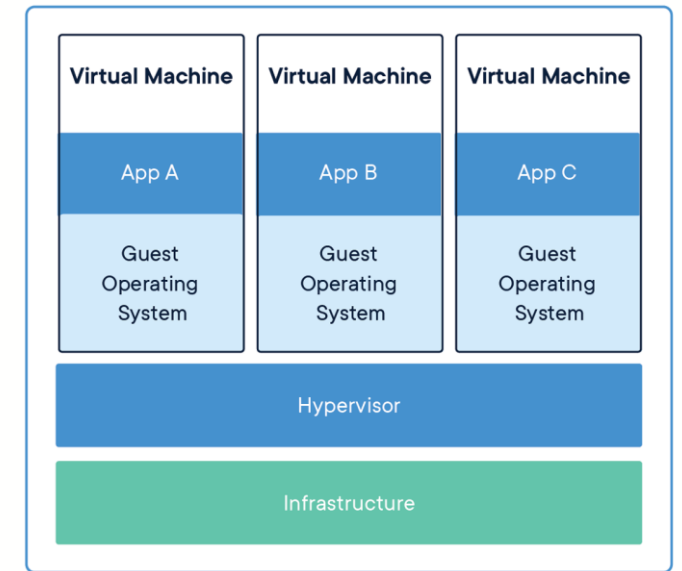
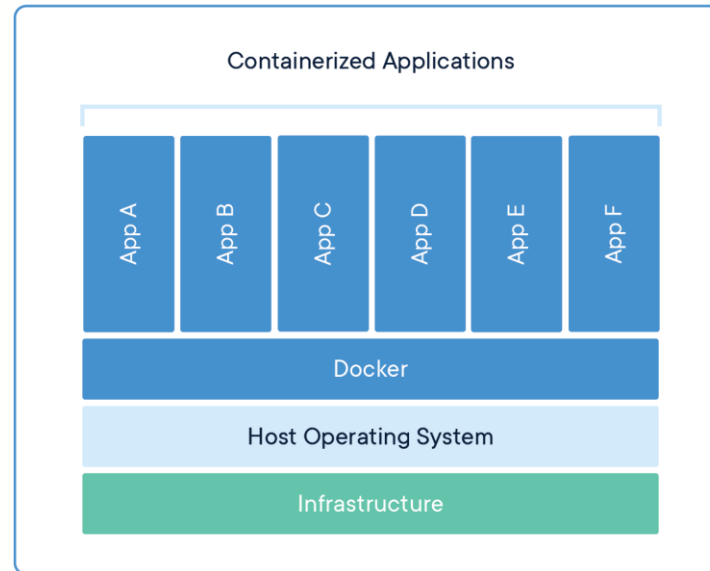
- Running a program in a VMs is less efficient than running it natively
 - This is because instructions to the Guest OS needs to be interpreted/translated for the Host OS
 - Hardware Virtualization support somewhat helps with this
- More resources are expended on the Hypervisor itself
- VM images get very large

For Big Data, we care about efficiency, so VMs are usually not a great option



Containers

- **Containers** are lightweight alternatives to virtual machines
 - Shares the OS with host system while still isolating software running inside
 - Created from "**images**", which are layers of modifications on top of a "base image"
 - Images are made available through **image repositories**
- Containers solve **application, library and executable** incompatibility
 - But **not necessary driver incompatibility**, as the container uses drivers installed on the host directly



<https://www.docker.com/resources/what-container>



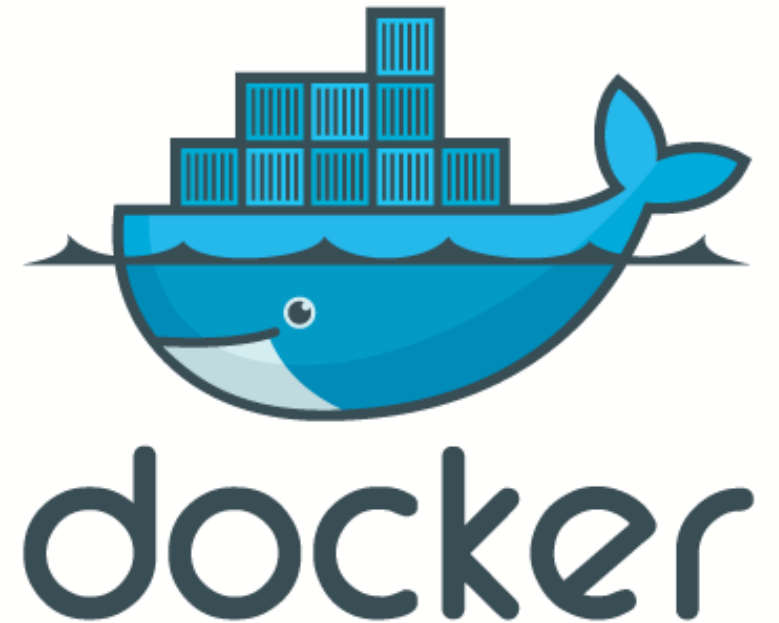
Containers

- You can consider a container to be a **sandbox environment** where you can run your code
- Each container runs a **single command** from the command line, and the container is **destroyed** once the command exits
 - This command may be a script that carries out a sequence of actions
 - Or it may start an application that will run continuously
- Any changes made to a container environment is lost when the container exits
 - So all program **output** must be **written** to **external mounted file stores** during run-time
- The container **image defines what software is available** to run, i.e. the libraries and executables that are installed, such as python or nvidia-smi
 - Your code and data will typically be mounted, not baked into the image



Docker

- **Docker** is a company that produces commercial solutions for container creation and management
- There are many types of containers, however, Docker designed the defacto-standard type, namely: **Docker Containers**
- This is backed by one of the largest container image repositories – **Dockerhub.io**
 - Provides ready-to-deploy images for the majority of application types





Containers

Advantages

- Containers are **more lightweight** than VMs, as each container does not need include a Guest OS
- Containers are **more efficient** than VMs, as they directly communicate with the machine hardware via the Host OS's drivers
- **Convenient** to get running, as image repositories have fostered a robust ecosystem
 - Provides applications that can be run 'out-of-the-box' (container)

Disadvantages

- Does not solve driver-level incompatibilities (where those are provided by the Host OS)
- Are less secure than VMs

Most (all?) Big Data Platforms provide their services in the form of containers that can be downloaded from DockerHub



University
of Glasgow

Course Coordinator
Dr. Richard McCreadie

Email: richard.mccreadie@glasgow.ac.uk
Room: SAWB 304

#UofGWorldChangers



@UofGlasgow